

Sessison16 - Java 8

1 Default Method - Static Method

Default Method

Static Method

Vi dụ

Cú pháp: **default** **[returnData]** **[methodName]** **(params)**{ Block Statements }

Cú pháp: **static** **[returnData]** **[methodName]****(params)**{ Block Statements }

Định nghĩa static và default method trong interface

```
// Example of an interface with a default method and a static method
interface MyInterface {
    // Default method
    default void displayDefault() {
        System.out.println("Default Method Executed");
    }

    // Static method
    static void displayStatic() {
        System.out.println("Static Method Executed");
    }
}
```

Sử dụng:

```
class MyClass implements MyInterface {
    // Overriding the default method
    public void displayDefault() {
        System.out.println("Overridden Default Method Executed");
    }
}

public class Main {
    public static void main(String[] args) {
        MyInterface myInterface = new MyClass();

        // Default method is called using object - overridden version
        myInterface.displayDefault();

        // Static method is called using interface name
        MyInterface.displayStatic();
    }
}
```

3 Lambda expression

Function Interface

Lambda expression

Vi dụ

là **interface** chỉ có duy nhất **một** phương thức trừu tượng

Sử dụng annotation **@FunctionalInterface** để kiểm tra xem 1 interface có phải là functional interface không

Java cung cấp các **functional interface** dùng sẵn trong gói **java.util.function**

Được sử dụng nhiều làm tham số của các **Stream API**

Là hàm ẩn danh, triển khai các functional Interface

Viết ít code hơn, hiệu quả hơn nhờ hỗ trợ thực hiện tuần tự (**sequential**) và song song (**parallel**) thông qua **Stream API**

Cú pháp: **(params)**->**{ Block Statements }**

```
//tùy chọn, nó đánh dấu lớp Drawable chỉ được có 1 method trừu tượng
@FunctionalInterface
interface Drawable {
    public void draw();
}

public class LambdaExpressionExample2 {
    public static void main(String[] args) {
        int width = 10;

        // sử dụng biểu thức lambda
        Drawable d2 = () -> {
            System.out.println("Drawing " + width);
        };
        d2.draw();
    }
}
```

2 Method Reference

Khái niệm

Lợi ích

Các loại

Vi dụ

Tham chiếu phương thức trong Java là cú pháp viết tắt cho biểu thức lambda chỉ chứa một lệnh gọi phương thức

Thường sử dụng như đối số truyền vào trong cácn Stream API

Cho phép truy cập trực tiếp tới các constructor hoặc method đã tồn tại của các lớp hoặc đối tượng mà không cần thực thi chúng

Không cần phải truyền đối số khi gọi phương thức

Giúp cú pháp ngắn gọn hơn

Tham chiếu đến một static method

Tham chiếu đến một instance method của đối tượng cụ thể

Tham chiếu đến một instance method của một đối tượng tùy y của một kiểu cụ thể

Tham chiếu đến một constructor

Lớp PrintService có 1 phương thức print hiển thị message được truyền vào

```
public class PrintService {
    public void print(String message) {
        System.out.println(message);
    }
}
```

Sử dụng StreamAPI forEach() để duyệt qua collection bằng lambda expression và method references

```
PrintService printService = new PrintService();
List<String> messages = Arrays.asList("Hello", "World", "Method", "References");

// using Lambda expression
messages.forEach(message -> printService.print(message));

// using method reference
messages.forEach(PrintService::print);
```

Class:staticMethod

object:instanceMethod

Class:instanceMethod

Class:new

4 Stream API

Phương thức stream

Làm việc với stream

stream(): trả về một stream sẽ được xử lý theo tuần tự

parallelStream(): trả về một stream song song, các xử lý sẽ được xử lý song song

Java Streams

Stream Source (An Array, collection, I/O channel etc.) -> Create Stream instance -> Intermediate Operations (Operation 1, Operation 2, ..., Operation N) -> Terminal Operation -> Operation Result (Aggregate result e.g. count, sum or collecting a collection etc.)

1. Tạo Stream

2. Thực hiện thao tác trung gian (Intermediate Operations)

3. Thực hiện thao tác đầu cuối (Terminal operation)

Tạo stream cho những kiểu dữ liệu primitive

Tạo stream từ các cấu trúc dữ liệu khác

Sử dụng filter()

Sử dụng skip(), limit()

Sử dụng map()

Sử dụng sorted()

Sử dụng foreach()

Sử dụng collect()

Sử dụng anyMatch(), allMatch(), noneMatch()

Sử dụng count()

Sử dụng min(), max()

Sử dụng summaryStatistics()

Sử dụng reduce()

5 Date Time API

Java Date and Time API

LocalDate 2025-04-14 Date only birthdays, holidays	LocalTime 10:30:45 Time only alarms, schedules	LocalDateTime 2025-04-14T10:30:45 Date + Time events, meetings	ZonedDateTime 2025-04-14T10:30:45-05:00 Date + Time + Zone international coordination
Formatting DateTimeFormatter "dd-MMM-yyyy" -> "14-Apr-2025"			
Time Differences Period: 2 years, 3 months Duration: 3600 seconds		Operations plusDays(), minusMonths() isBefore(), isAfter()	

Các thao tác làm việc chính

6 Optional

Lợi ích

Java.util.Optional

Sử dụng kiểm tra một biến có giá trị tồn tại giá trị hay không

Tránh kiểm tra null quá nhiều và tránh lỗi NullPointerException

Java 8 Optional Class Methods & Description

Methods	What it does?
empty()	Creates an empty Optional object.
of()	Creates an Optional object with specified non-null value.
ofNullable()	Creates an Optional object with specified value if it is non-null, otherwise returns empty Optional.
get()	Returns value present in Optional object. If the value is absent, throws NoSuchElementException.
orElse()	Returns value if present, otherwise returns supplied value.
ifPresent()	Performs specified action if value is present, otherwise no action taken.
isPresent()	Checks whether value is present or not.
orElseGet()	Returns value if present, otherwise returns result of specified supplier.
orElseThrow()	Returns value if present, otherwise throws exception created by specified supplier.
map()	Applies given mapping function if value is present. Returns Optional containing the result. If result is null, returns empty Optional.
flatMap()	It is used when mapper function returns another Optional as a result and you don't want to wrap it in another Optional.
filter()	If value is present and matches with given predicate then it returns Optional containing the result. Otherwise returns empty Optional.